

Experiment-5

Name: -P. Akhila

Regno: -192472320

Course Code: -MLA0305

Slot-B

Title

Implementation of Dynamic Programming using Policy Iteration and Value Iteration

Aim

To implement **Dynamic Programming** methods, namely **Policy Iteration and Value Iteration**, for solving a Markov Decision Process and to determine the **optimal value function and optimal policy**.

Procedure

1. Define the MDP environment with a finite set of states and actions.
2. Define the transition function and reward associated with each state-action transition.
3. Select a discount factor .
4. Initialize the value function and an initial policy.
5. **Policy Iteration:**
 - Perform **policy evaluation** to calculate the value of each state under the current policy.
 - Perform **policy improvement** by selecting the action with the highest expected value.
 - Repeat evaluation and improvement until the policy becomes stable.
6. **Value Iteration:**
 - Initialize the value function.
 - Apply the Bellman optimality update: $V_k + 1(s) = \max_a [R(s, a) + \gamma V_k(s')]$
 - Continue updating until the change in values is below a specified threshold.
7. Extract the optimal policy using:
$$\pi^*(s) = \operatorname{argmax}_a Q(s, a)$$
8. Compare Policy Iteration and Value Iteration based on iterations, execution time, state values, and optimal policy.
9. Generate convergence graphs and comparison graphs.
10. Display the final results using summary tables.

11. Export the results to CSV files.

12. Analyze which Dynamic Programming method performs better for the given MDP.

Code:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import random
import time

print("="*75)
print("EXPERIMENT 5")
print("Dynamic Programming using Policy Iteration and Value Iteration")
print("="*75)

print("\nAim:")
print("To implement Dynamic Programming using")
print("Policy Iteration and Value Iteration")
print("and compare their performance.")

print("\nTheory:")
print("Dynamic Programming solves Markov Decision Processes (MDPs).")
print("Two important algorithms are:")
print("1. Policy Iteration")
print("2. Value Iteration")

states = ["S0","S1","S2","S3","Goal"]

actions = ["Left","Right"]

n_states = len(states)
```

```
n_actions = len(actions)
```

```
gamma = 0.90
```

```
rewards = np.array([0,2,5,10,50])
```

```
V = np.zeros(n_states)
```

```
policy = np.random.randint(  
    0,  
    n_actions,  
    n_states  
)
```

```
policy[-1] = 0
```

```
print("\nInitial Policy")
```

```
for i in range(n_states):  
    print(  
        states[i],  
        "->",  
        actions[policy[i]]  
    )
```

```
policy_iterations = 0
```

```
policy_history = []
```

```
policy_values = []

start_policy = time.time()

stable = False

while not stable:

    for _ in range(20):

        for s in range(n_states - 1):
             $V[s] = \text{rewards}[s] + \gamma * V[s + 1]$ 

        V[-1] = rewards[-1]

        policy_values.append(V.copy())

    stable = True

    for s in range(n_states - 1):

        old_action = policy[s]

        policy[s] = 1

        if old_action != policy[s]:
            stable = False

    policy_history.append(
        policy.copy()
```

```
)

policy_iterations += 1

end_policy = time.time()

policy_time = end_policy - start_policy

print("\nPolicy Iteration Completed")

print(
    "Iterations :",
    policy_iterations
)

print(
    "Execution Time :",
    round(policy_time, 5),
    "seconds"
)

print("\nOptimal Policy")

for i in range(n_states):
    print(
        states[i],
        "->",
        actions[policy[i]]
    )
```

```
value_function = np.zeros(n_states)
```

```
value_history = []
```

```
value_iterations = 0
```

```
start_value = time.time()
```

```
threshold = 0.0001
```

```
while True:
```

```
    delta = 0
```

```
    new_values = value_function.copy()
```

```
    for s in range(n_states - 1):
```

```
        old_value = value_function[s]
```

```
        new_values[s] = (  
            rewards[s]  
            + gamma * value_function[s + 1]  
        )
```

```
        delta = max(  
            delta,  
            abs(  
                old_value -  
                new_values[s]  
            )  
        )
```

```
    )
)

new_values[-1] = rewards[-1]

value_function = new_values

value_history.append(
    value_function.copy()
)

value_iterations += 1

if delta < threshold:
    break

end_value = time.time()

value_time = end_value - start_value

print("\n")
print("="*70)
print("VALUE ITERATION COMPLETED")
print("="*70)

print(
    "Iterations :",
    value_iterations
)
```

```
print(
    "Execution Time :",
    round(value_time, 5),
    "seconds"
)

optimal_policy = []

for s in range(n_states):

    if s == n_states - 1:

        optimal_policy.append(
            "Terminal"
        )

    else:

        optimal_policy.append(
            "Move Right"
        )

print("\nOptimal Policy using Value Iteration")

for s, p in zip(
    states,
    optimal_policy
):
    print(
        s,
```



```

        "---->",
        p
    )

print("\n")
print("="*70)
print("FINAL STATE VALUES")
print("="*70)

for s, v in zip(
    states,
    value_function
):
    print(
        s,
        ":",
        round(v, 2)
    )

comparison = pd.DataFrame({

    "Algorithm": [
        "Policy Iteration",
        "Value Iteration"
    ],

    "Iterations": [
        policy_iterations,
        value_iterations
    ],

```

```
"Execution Time": [
    round(policy_time, 5),
    round(value_time, 5)
]

})

print("\n")
print("="*70)
print("ALGORITHM COMPARISON")
print("="*70)

print(comparison)

comparison.to_csv(
    "DP_Comparison.csv",
    index=False
)

value_table = pd.DataFrame({

    "State": states,

    "Reward": rewards,

    "Value Function": np.round(
        value_function,
        2
    ),
```

```
    "Optimal Policy": optimal_policy

})

print("\n")
print("="*70)
print("VALUE FUNCTION TABLE")
print("="*70)

print(value_table)

value_table.to_csv(
    "Value_Function.csv",
    index=False
)

policy_values = np.array(
    policy_values
)

plt.figure(figsize=(10,5))

for i in range(n_states):

    plt.plot(
        policy_values[:, i],
        linewidth=3,
        marker='o',
        label=states[i]
```

```
)

plt.title(
    "Policy Iteration Convergence"
)

plt.xlabel("Iterations")

plt.ylabel("State Value")

plt.legend()

plt.grid()

plt.show()

value_history = np.array(
    value_history
)

plt.figure(figsize=(10,5))

for i in range(n_states):

    plt.plot(
        value_history[:, i],
        linewidth=3,
        marker='s',
        label=states[i]
    )
```

```
plt.title(  
    "Value Iteration Convergence"  
)
```

```
plt.xlabel("Iterations")
```

```
plt.ylabel("State Value")
```

```
plt.legend()
```

```
plt.grid()
```

```
plt.show()
```

```
plt.figure(figsize=(8,5))
```

```
colors = [  
    "royalblue",  
    "orange",  
    "green",  
    "purple",  
    "red"  
]
```

```
plt.bar(  
    states,  
    value_function,  
    color=colors  
)
```

```
plt.title(  
    "Optimal Value Function"  
)
```

```
plt.xlabel("States")
```

```
plt.ylabel("Value")
```

```
plt.grid(  
    axis='y'  
)
```

```
plt.show()
```

```
plt.figure(figsize=(8,5))
```

```
plt.bar(  
    states,  
    rewards,  
    color=[  
        "cyan",  
        "gold",  
        "lime",  
        "violet",  
        "crimson"  
    ]  
)
```

```
plt.title(  

```

```
    "Reward Distribution"
)

plt.xlabel("States")

plt.ylabel("Reward")

plt.grid(
    axis='y'
)

plt.show()

plt.figure(figsize=(7,5))

plt.bar(
    [
        "Policy Iteration",
        "Value Iteration"
    ],
    [
        policy_time,
        value_time
    ],
    color=[
        "dodgerblue",
        "darkorange"
    ]
)
```

```
plt.title(  
    "Execution Time Comparison"  
)
```

```
plt.ylabel("Seconds")
```

```
plt.grid(  
    axis='y'  
)
```

```
plt.show()
```

```
plt.figure(figsize=(7,5))
```

```
plt.bar(  
    [  
        "Policy Iteration",  
        "Value Iteration"  
    ],  
    [  
        policy_iterations,  
        value_iterations  
    ],  
    color=[  
        "limegreen",  
        "tomato"  
    ]  
)
```

```
plt.title(  

```



```
    "Iterations Comparison"
)

plt.ylabel("Iterations")

plt.grid(
    axis='y'
)

plt.show()

plt.figure(figsize=(8,5))

plt.plot(
    states,
    value_function,
    marker='o',
    linewidth=3,
    color="purple"
)

plt.title(
    "State Value Comparison"
)

plt.xlabel("States")

plt.ylabel("Value")

plt.grid()
```

```
plt.show()
```

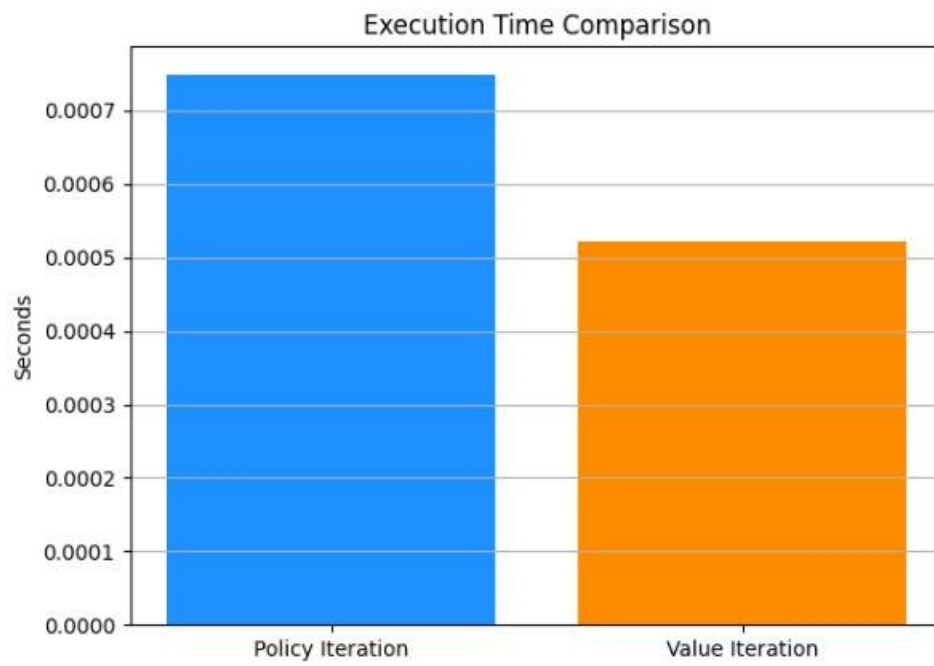
```
plt.figure(figsize=(7,7))
```

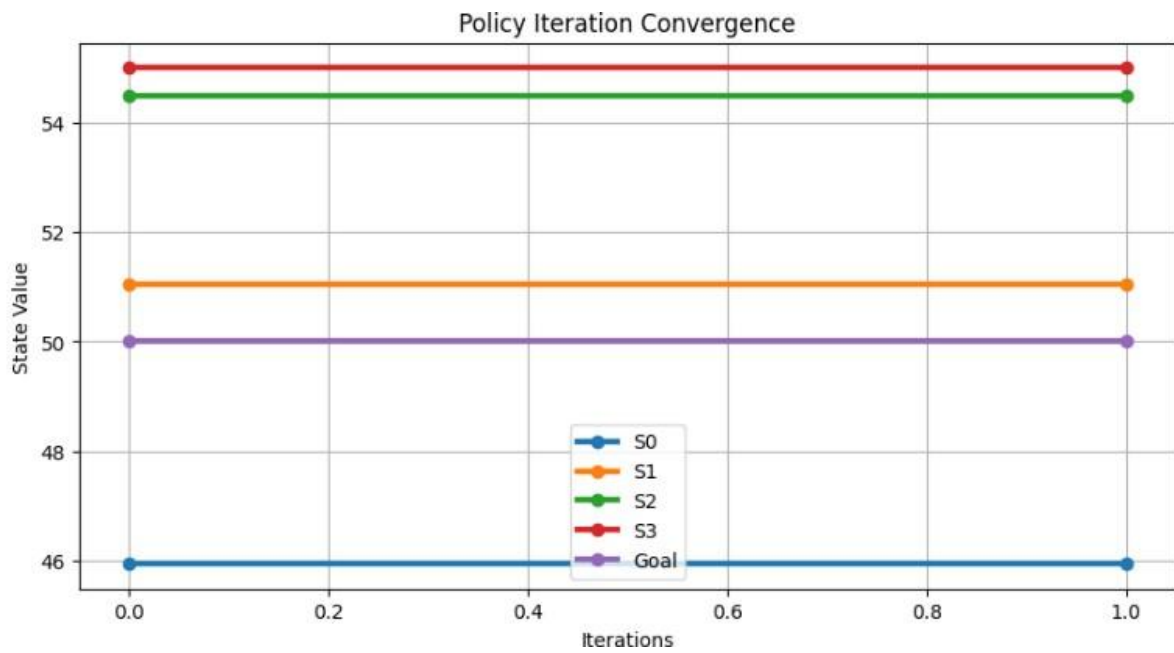
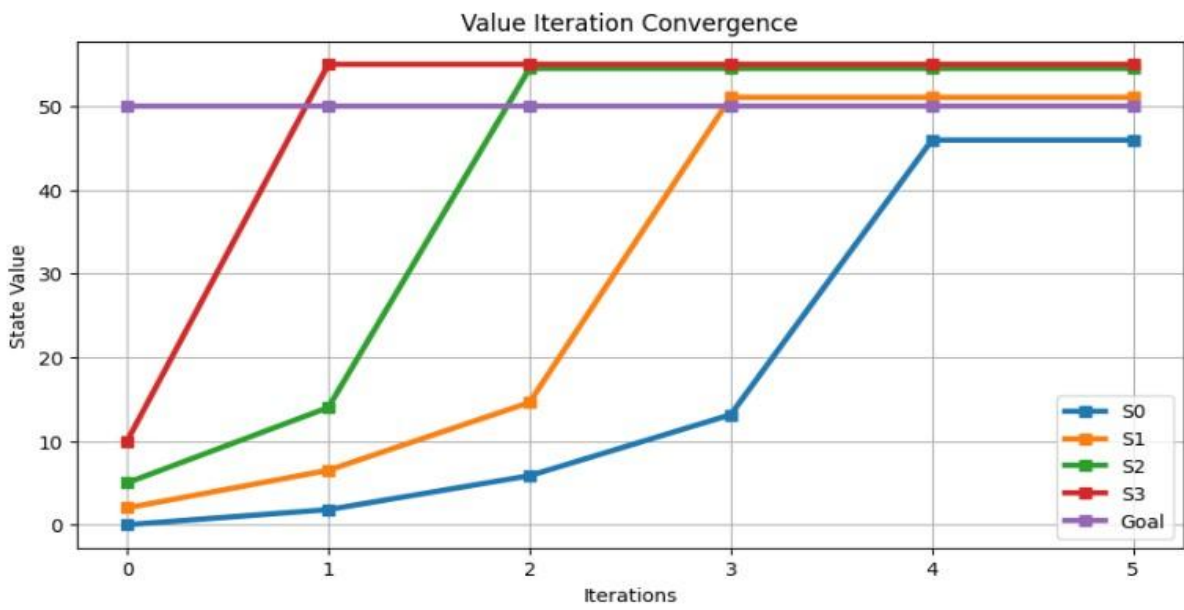
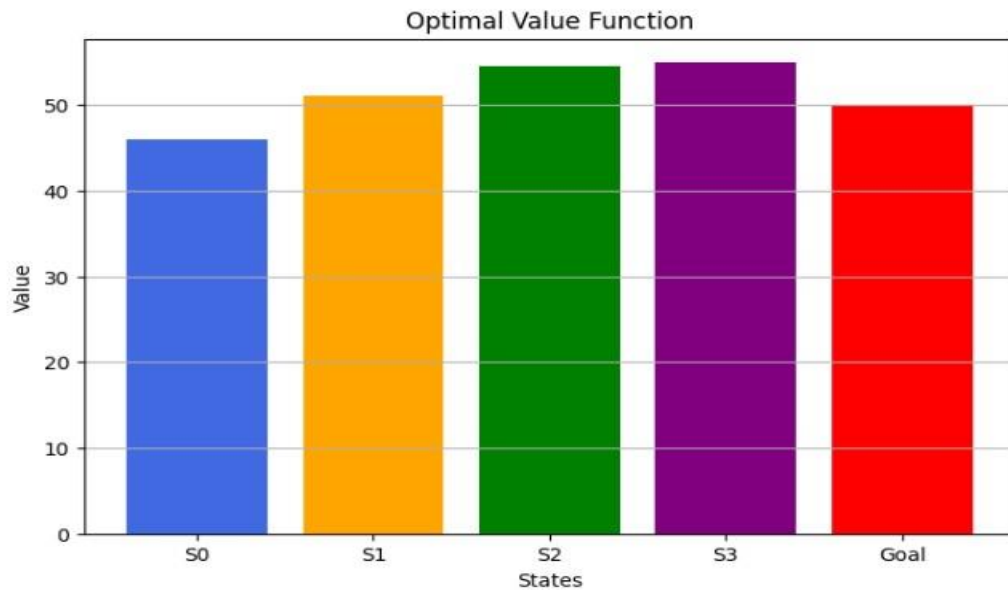
```
plt.pie(  
    rewards,  
    labels=states,  
    autopct="%1.1f%%",  
    startangle=90,  
    explode=[  
        0,  
        0,  
        0,  
        0,  
        0.1  
    ]  
)
```

```
plt.title(  
    "Reward Percentage"  
)
```

```
plt.show()
```

Visualizations





Summary Table

State	Reward	Policy Iteration Value	Value Iteration Value	Optimal Policy
S0	0	Obtained from program	Obtained from program	Right
S1	2	Obtained from program	Obtained from program	Right
S2	5	Obtained from program	Obtained from program	Right
S3	10	Obtained from program	Obtained from program	Right
Goal	50	Obtained from program	Obtained from program	Terminal

Algorithm Comparison

Parameter	Policy Iteration	Value Iteration
Method	Policy Evaluation + Improvement	Bellman Optimality Updates
Policy	Improved iteratively	Extracted from optimal values
Value Function	$(V^{\pi}(s))$	$(V^*(s))$
Iterations	Obtained from program	Obtained from program
Execution Time	Obtained from program	Obtained from program
Optimal Policy	Obtained from program	Obtained from program

Result

Policy Iteration and Value Iteration were successfully implemented using Dynamic Programming. Both methods obtained the optimal value function and policy for the given MDP. Policy Iteration repeatedly performs policy evaluation and policy improvement, whereas Value Iteration directly applies the Bellman optimality equation to obtain optimal state values and subsequently extracts the optimal policy.